

Can It Edit? Evaluating the Ability of LLMs for Code Editing

Federico Cassano¹, Luisa Li¹, Akul Sethi¹, Noah Shinn¹, Abby Brennan-Jones²,
Anton Lozhkov³, Carolyn Jane Anderson², Arjun Guha^{1,4}

Northeastern University¹, Wellesley College², Hugging Face³, Roblox⁴

Motivation

- Code LLMs have become an increasingly essential tool for software engineering and research.
- Significant research has been devoted to code synthesis (HumanEval, MBPP), and existing code editing tasks focus primarily on a narrow scope of code edits, such as bug fixing.
- We aim to bridge that gap to evaluate a larger variety of tasks.

Why Code Editing?

- Data from LMSys, which feature in-the-wild conversations with chatbots, has 4,188 conversations containing code, with 19% of those involving code edits.
- Code editing ability is also essential for AI-focused code editors such as Cursor, Copilot Chat, or ChatGPT Advanced Data Analysis.



GitHub
Copilot

```

- class C4(nn.Module):
+ class C8(nn.Module):
- """Represents the cyclic group C4,
+ """Represents the cyclic group C8,
  where each element represents a discrete
  rotation."""

  def __init__(self):
      super().__init__()

  def size(self):
      """Outputs the size of this group."""
-   return 4
+   return 8

  def elements(self):
      """Returns all the elements of this
      group"""
-   return torch.tensor([0., np.pi/2, np.pi,
-   3*np.pi/2])
+   d = np.pi / 4
+   return torch.tensor([0., d, d*2, d*3, d*4,
+   d*5, d*6, d*7])

```

group_theory problem from CanItEdit

Instruction	Type
Edit the C4 class, which represents rotations of 0, 90, 180 and 270 degrees, to represent the class C8, which represents rotations of 0, 45, 90, 135, 180, 225, 270 and 315 degrees.	Descriptive
Edit the C4 class and its methods to represent the C8 group instead	Lazy

The *CanItEdit* Dataset

- 105 meticulously constructed code editing challenges with comprehensive test suites in Python, which include evaluation of non-functional requirements.
- Broad spectrum of domains such as data structures, algorithms and mathematics, which require domain specific knowledge.
- Categorized into corrective, perfective and adaptive¹

CanItEdit Dataset Statistics	
Total Tasks	105
Topics	
Data Structures & Algorithms	39
Language Processing	21
Mathematics	25
Data Science	10
Miscellaneous	10
External Library Usage	
NumPy (13), Pandas (6), SciPy (3), scikit-learn (3), PyTorch (3), Z3 (2), autograd (2), Flask (1), vLLM (1)	

1. E. Burton Swanson. The dimensions of maintenance. In Proceedings of the 2nd International Conference on Software Engineering, ICSE '76, pp. 492–497, Washington, DC, USA, 1976. IEEE Computer Society Press.

Test Suites that Evaluate Non-Functional Requirements

- `minimax_to_alphabeta`: Instructs the model to augment the minimax algorithm with alpha-beta pruning for a Connect 4 game. A solution is successful if the improved algorithm yields the correct next move with less positions evaluated.
- `oop_refactor`: Instructs the model to abstract the code given into an object-oriented version of itself. Correctness is achieved if the code is functionally correct, and the class inheritance is appropriately implemented.

Evaluation Metrics

- *Pass@k*: The likelihood that at least one successful edit was made from k attempts.
- *ExcessCode*: Quantifies the presence of unnecessary code changes, indicated by the fraction of changed lines not covered by the test suite.

```
class Tensor:

    def __init__(self, matrix):
        self.matrix = matrix

    def m(self):
        return len(self.matrix)

    def n(self):
        return len(self.matrix[0])

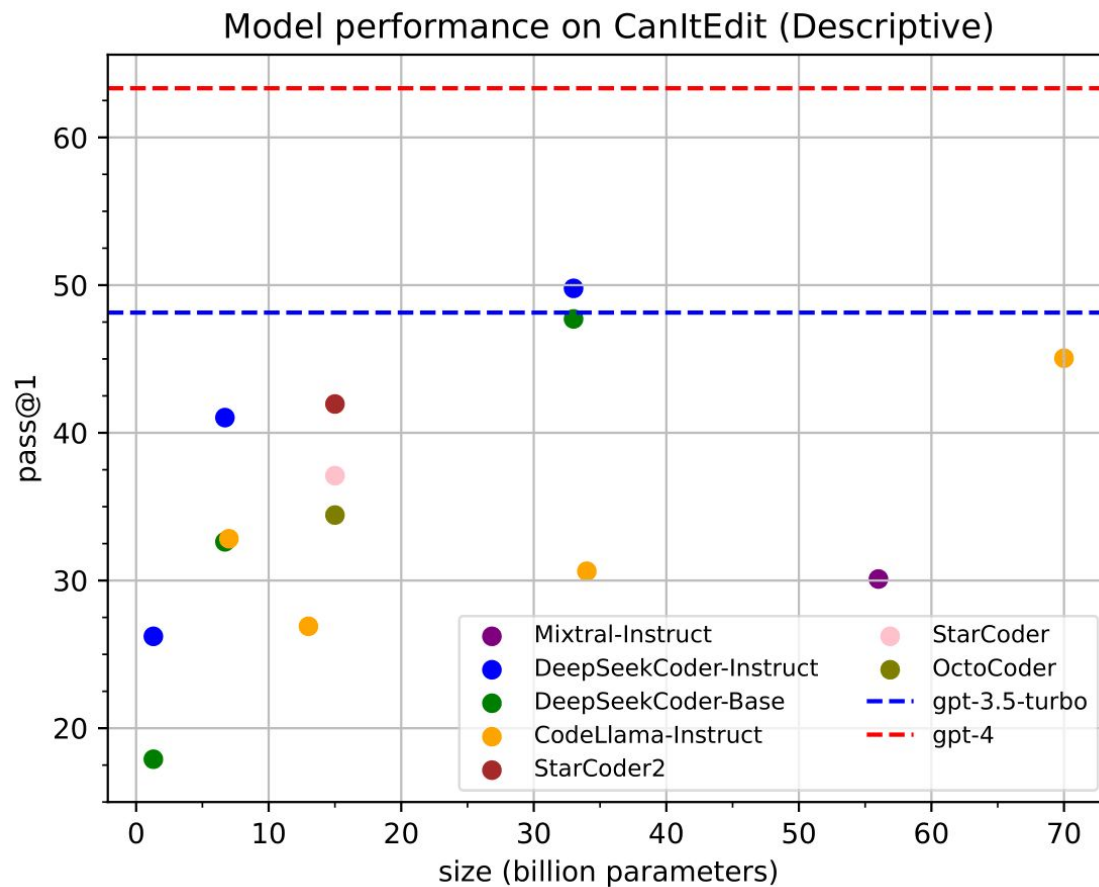
    def relu(self):
        for i in range(self.m()):
            for j in range(self.n()):
                self.matrix[i][j] = max(0, self.matrix[i][j])

    def flatten(self):
       sofar = []
        - for i in range(self.n()):
        -     for j in range(self.m()):
        -         sofar.append(self.matrix[j][i])
        + for i in range(self.m()):
        +     for j in range(self.n()):
        +         sofar.append(self.matrix[i][j])
        return sofar

    + def unflatten(self, flat):
    +     self.matrix = []
    +     for i in range(self.n()):
    +         self.matrix.append([])
    +         for j in range(self.m()):
    +             self.matrix[i].append(flat[i*self.n() + j])
```

Instruction: Change 'flatten' to flatten lists left to right, top down

Evaluation

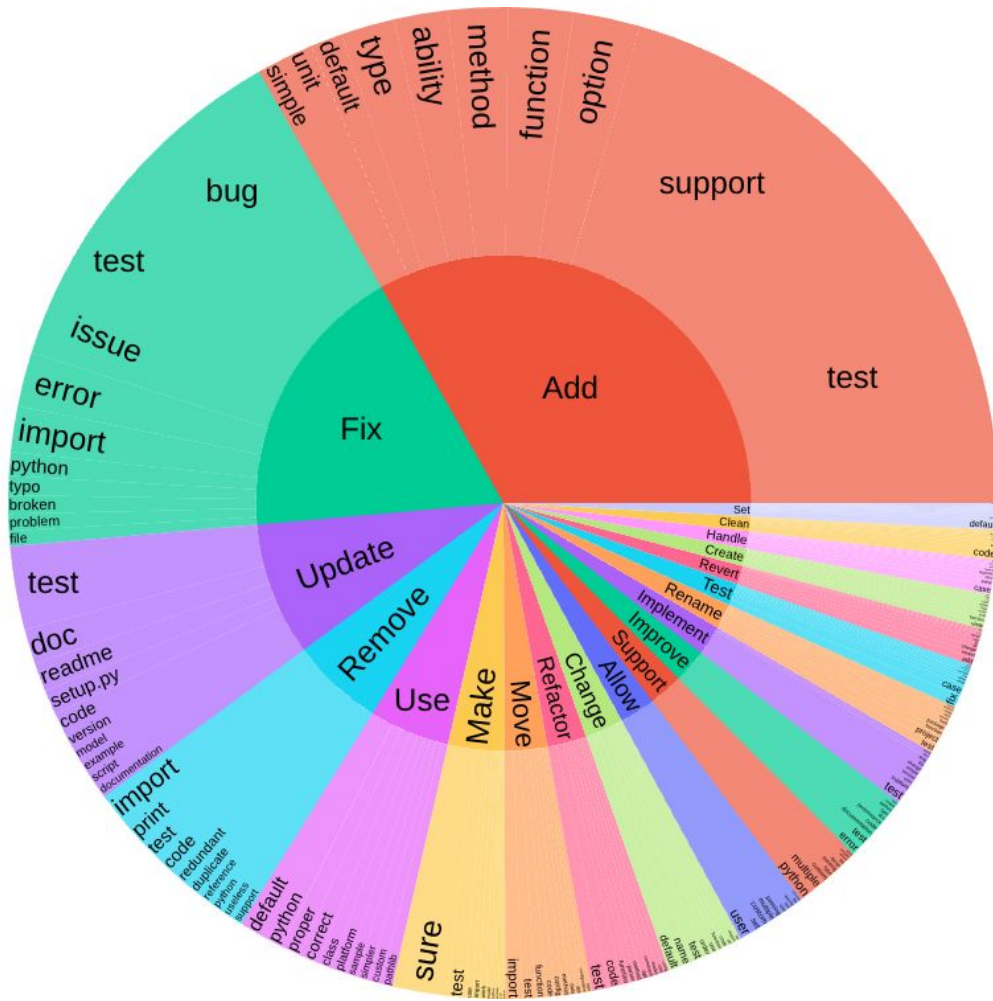


EditPackFT

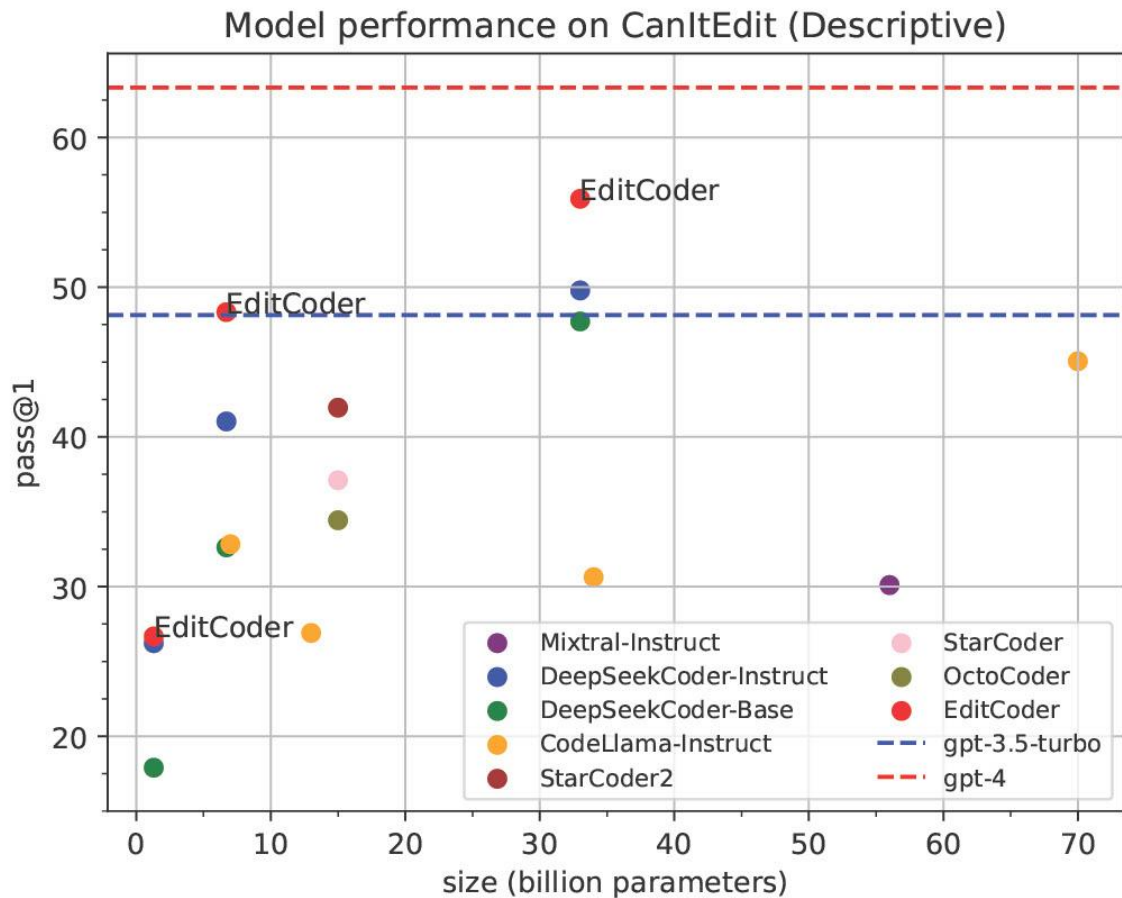


1. Niklas Muennighoff. Octopack: Instruction tuning code large language models. In NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following, 2023. URL <https://openreview.net/forum?id=CjrPqvUX>

Verbs



EditCoder



Contributions

1. We introduce CanItEdit, a dataset of instructional code editing problems
2. We propose a novel metric, ExcessCode, for quantifying the typical volume of unwanted code produced by a model when generating the correct code edits
3. We perform a thorough evaluation of the latest Code LLMs in the context of code editing, providing insights into their current capabilities
4. We present a specially tailored training dataset for fine-tuning a model to code edit

Thank you!



Read Paper



Use Benchmark